



天数智芯
Iluvatar CoreX

天数智芯

编译器使用指南

版本：V4.3.0 (x86)|V4.3.6 (ARM)

日期：2025.6.30

适用产品：天垓 150| 智铠系列

声明

版权与保密声明

本文档（属未发表作品）版权归属于天数智芯全权所有，且本声明不构成对本文档已发表的承认。本文档所载全部信息均为天数智芯保密信息（包括但不限于天数智芯技术信息、经营信息、专属于接收方的未公开资料等）。未经天数智芯事先书面许可，严禁任何形式的使用、复制、传播、转录或翻译。

免责声明

天数智芯可以随时对本文档或本文档中描述的产品进行改进和/或更改。本文档包括与天数智芯产品有关的信息，作为说明典型应用的一种方式，因此，不一定提供足以进行生产设计的完整信息。对于本文档中内容的准确性或完整性，天数智芯不做任何陈述或保证。

联系方式

地址：上海闵行区陈行公路 2168 号 3 幢

电话：021-68886607

网址：www.iluvatar.com

目录

1 编译器使用指南	4
1.1 修订记录	4
1.2 概览	4
1.3 快速入门	5
1.4 示例	5
1.5 宏	6
1.6 Function Call	6
1.6.1 使用方式	7
1.6.2 操作示例	7
1.7 高级用法	9
1.7.1 编译阶段	9
1.7.2 输入和输出文件	10
1.7.3 编译步骤示例	10
1.7.3.1 1. 生成 LLVM IR	10
1.7.3.2 2. 将 LLVM IR 编译到 OBJ 文件	11
1.7.3.3 3. 将 OBJ 文件链接到 .cubin	11
1.7.3.4 4. 从 .cubin 文件中创建 .fatbin 文件	11
1.7.3.5 5. 编译主机部分的代码并链接到 .fatbin	12
1.7.3.6 6. OBJ 文件链接到天数智芯 CUDA 运行环境	12
1.8 编译选项	12
1.8.1 clang 和 clang++ 选项	12
1.8.2 llc 选项	13
1.8.3 lld 选项	14
1.8.4 fatbinary 选项	14
1.9 参考	15

1 编译器使用指南

1.1 修订记录

- COREX01-BI430-UG03-00: 2025/6/30
 - 宏中, 修改 GPU 架构参数宏为 `__IVCORE_ARCH__`
 - 增加 **Function Call** 功能说明
 - **clang** 和 **clang++** 选项 新增 **--cubin** 选项, 用于将所有 .cu 输入文件编译为仅包含设备代码的 .cubin 文件
- COREX01-BI420-UG03-00: 2025/1/24
 - 更新: 当 **--cuda-gpu-arch=<value>** 的值为 **ivcore11** 时, 指代天核 150 加速卡或智铠 100 加速卡或智铠 50 加速卡
 - **编译步骤示例** 的 1. 生成 LLVM IR 中, 更新为 CUDA 代码部分生成文本 LLVM IR 文件的指令, 在指令中增加 **-O3** 选项。如果不加 **-O3** 选项, 编译过程会卡住
- COREX01-BI411SP1-UG03-00: 2024/12/3
 - 修改天核 150 **--cuda-gpu-arch=<value>** 选项的默认值为 **ivcore11**
- COREX01-BI411-UG03-01: 2024/11/18
 - 补充说明 **-I/usr/local/corex/include** 是编译器默认包含路径
- COREX01-BI411-UG03-00: 2024/8/26

无内容更新
- COREX01-BI410-UG03-00: 2024/6/30

无内容更新
- COREX01-BI340-UG03-01: 2024/6/4

本次文档发布内容与上一次发布 (COREX01-BI340-UG03-00) 相比, 有以下更新:

 - **编译选项** 新增备注, 提示用户 Clang 和 nvcc 的默认选项可能存在少量不同, 编译选项请按需调整

1.2 概览

天数智算软件栈提供的编译器是针对天数智芯硬件架构设计和优化的。由于天数智芯编译器是基于开源 LLVM 项目开发的, 因此调用天数智芯编译器的命令是 **clang** 或 **clang++**。LLVM 项目包括 Clang 子项目, Clang 是 LLVM 原生的 C 语言家族 (C, C++, Objective C/C++, CUDA 等) 的编译器。

本指南旨在强调天数智芯编译器的用法。关于 Clang 等 LLVM 工具的一般用法, 请参考 LLVM 官方文档。

1.3 快速入门

1. 检查您是否已在使用天数智算软件栈提供的 Clang 编译器。运行结果应显示 `${ILUVATAR_SOFTWARE_ROOT}/bin/clang++`，此处的 `${ILUVATAR_SOFTWARE_ROOT}` 指的是天数智算软件栈的安装路径。默认安装路径是 `/usr/local/corex/`。

```
$ which clang++
```

2. 从 `.cu` 源文件构建可执行文件，以编译使用了 C++11 高级特性的 `example.cu` 文件生成 `example` 程序为例：

```
$ clang++ -std=c++11 example.cu -o example -lcudart
```

具体的编译实例，请参见[示例](#)。

1.4 示例

以下示例是以编译天数智算软件栈提供的 `saxpy.cu` 示例代码为例，构建名为 `saxpy` 的可执行文件。假设使用默认安装路径 `/usr/local/corex/`，请注意执行示例命令需要您对 `/usr/local/corex/` 有写权限。

1. 切换到示例所在的目录。

```
$ cd /usr/local/corex/examples/cuda
```

2. 运行 `clang++` 命令及相应的必需和可选参数，生成名为 `saxpy` 的可执行文件。

```
$ clang++ -std=c++11 -Wall -lcudart saxpy.cu -o saxpy
```

Note

如果您安装了其他版本的 Clang 编译器，请确保您调用的 Clang 编译器路径是 `/usr/local/corex/bin/`。如需要了解更多信息，请参阅[快速入门](#)。

示例中的选项详细说明如下：

- `-std=c++11`: 可选项，本示例中 `saxpy.cu` 实际未使用 C++11 高级特性。如您的代码中使用了某些 C++ 高级特性，则需要使用 `-std`，具体可参考 [C++ Support in Clang](#)。
- `-Wall`: 可选项。启用某些用户认为有问题的构造器的所有警告。这也启用了针对特定语言的警告。
- `-lcudart`: 必选项。确保链接 CUDA 运行库；否则，会出现一些无法解析的符号。

其它相关命令选项：

- `--cuda-gpu-arch=<value>`: 可选项。默认是 `--cuda-gpu-arch=ivcore11`，指代天垓 150 加速卡或智铠 50 加速卡或智铠 100 加速卡。
- `-o <your output name>`: 可选项。用于设置输出的文件名。默认是 `a.out`。

编译器默认开启了以下选项：

- `-I/usr/local/corex/include`: 编译器默认的包含路径，用于在预处理期间设置搜索头文件的目录。
- `--cuda-path=/usr/local/corex`: 指定天数智算软件栈的路径。
- `-L/usr/local/corex/lib64`: 指定库文件路径。

Tip

您可参考 `/usr/local/corex/examples/cuda/Makefile` 路径下的文件，您可以使用 **make** 完成编译：

```
$ make VERBOSE=1
```

您可以根据实际需求对 Makefile 进行自定义。

1.5 宏

__ILUVATAR__ 当使用天数智芯编译器为天数智芯平台进行设备代码编译时，该宏被定义。你可以使用 **__ILUVATAR__** 来确定是针对天数智芯平台进行编译，或是针对其他平台进行编译。

此外，当该宏被定义时，wrap size 设置为 64，否则，wrap size 为 32。

__IVCORE_ARCH__ GPU 架构参数。您可以使用该宏来确定算力。该宏的值由 **--cuda-gpu-arch=< value >** 选项定义。对于天数智芯编译器，**--cuda-gpu-arch** 的默认值为 **ivcore11**，表示天垓 150 加速卡或智铠 50 加速卡或智铠 100 加速卡。

1.6 Function Call

Function Call 功能，即可通过编译器配置，选择启用显式函数调用方式 (**__noinline__**) 替代默认的内联函数调用方式 (**__forceinline__**)，解决函数调用过程中，由于内联展开导致代码膨胀 (Code Bloat) 的问题，适用于存在大型函数或频繁调用复杂函数 CUDA 程序的场景。

在编译 CUDA 程序时，**__device__** 函数通常会被默认标记为 **__forceinline__** (即，函数调用在编译时被内联展开)。内联函数调用方式可消除调用开销，提升编译性能；但同时可能会导致代码膨胀，每个函数调用点都会复制一份函数体；当大量 device 函数被内联时，编译出的 binary 体积会急剧增大；从而导致项目的可维护性差，Inline 的代码难以逐步追踪，且堆栈信息中不会保留函数调用关系，增加了问题定位的复杂度。

通过 Function Call 功能配置，可在 **高频调用或调用点极多**的场景下，将“代码复制”(**__forceinline__**) 替换为“代码复用”(**__noinline__**) 方式，用于提升代码编写及维护效率，减少代码冗余。

显式函数调用方式 (**__noinline__**) 虽然可避免代码膨胀的问题，但同时也会出现函数调用开销增加，编译器优化受限等问题；因此在实际使用过程中，请根据实际场景需求执行调用方式的切换。

建议优先选择默认内联函数调用方式 (“__forceinline__”)，仅在出现以下场景时可考虑启用显式函数调用方式 (“__noinline__”)：

- 大型复杂函数调用
- 高频函数调用
- 多内核共享通用函数调用
- 函数调试阶段
- 代码体积过大导致编译失败或运行异常

Note

目前 Function Call 功能已在智铠加速卡上实现支持，且在 ixCUB 上通过正确性测试。

1.6.1 使用方式

天数智算软件栈提供以下三种可以通过 CUDA 程序实现的配置操作：

- 在 **编写** CUDA 程序时，为不需要 Inline 的 device 函数增加 `__attribute__((noinline))` 参数。
- 在 **编译** CUDA 程序时，由于编译器实现限制，请关闭 Tail call 和 Sibling call 相关优化，具体配置如下：
 - 如使用 clang 编译 CUDA 文件 (.cu)，需增加如下选项：

```
-fno-optimize-sibling-calls -mllvm -iluvatar-function-calls=true
```

- 如使用 llc 编译 LLVM IR，需增加如下选项：

```
-disable-tail-calls -mllvm -iluvatar-function-calls=true
```

选项说明：

- * `-fno-optimize-sibling-calls`：表示关闭 LLVM 框架的 Tail Call 优化，该优化不适用于 GPU
- * `-mllvm -iluvatar-function-calls=true`：表示开启 Function Call 优化
- * `-disable-tail-calls`：表示关闭编译器里的 Tail Call 优化

- 此外,也可从 CUDA 程序中关闭相关优化配置,给相应函数增加 `__attribute__((disable_tail_calls))` 配置选项。

注意：由于 CUDA 程序的限制，智铠加速卡支持 Function Call，但不支持 Indirect Call、可变参数列表和 Lib Call；因此在 CUDA 程序中请避免出现这几种写法，以免出现报错。

1.6.2 操作示例

在执行带有 Function Call 的程序驱动时，请完成以下 **关闭 UMD 的 lazy load** 的配置操作，以免程序无法正常运行。

```
#include <iostream>

// don't forget attr device func with noinline
__attribute__((noinline)) __device__ int addDevice(int a, int b) {
    return a + b;
}

__global__ void addKernel(int* results, int base, int increment) {
    int idx = threadIdx.x;

    int a = base + idx;
    int b = increment + idx;
    results[idx] = addDevice(a, b);
}

/*
Please add the -fno-optimize-sibling-calls parameter when compiling with clang,
```

and add UMD_CUDAMODULELOADING=0 when executing.

```
As: clang++ -mllvm -iluvatar-function-calls=true -fno-optimize-sibling-calls *.cu -o
↳ a.out
==>LD_LIBRARY_PATH=.. UMD_CUDAMODULELOADING=0 ./a.out
*/
```

示例 2:

```
#include <stdio.h>

__attribute__((noinline)) __device__ float device2(float a, float b) {
    return a * a + b * b;
}

__attribute__((noinline)) __device__ float device1(float x, float y) {
    float temp = device2(x, y);
    return temp + x * y;
}

__global__ void global_kernel(float* input, float* output, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        output[idx] = device1(input[idx], input[idx] + 1.0f);
    }
}

int main() {
    const int N = 1024;
    float *d_input, *d_output;
    float h_input[N], h_output[N];

    for (int i = 0; i < N; ++i) {
        h_input[i] = static_cast<float>(i);
    }

    cudaMalloc(&d_input, N * sizeof(float));
    cudaMalloc(&d_output, N * sizeof(float));

    cudaMemcpy(d_input, h_input, N * sizeof(float), cudaMemcpyHostToDevice);

    int blockSize = 256;
    int numBlocks = (N + blockSize - 1) / blockSize;
    global_kernel<<<numBlocks, blockSize>>>>(d_input, d_output, N);

    cudaMemcpy(h_output, d_output, N * sizeof(float), cudaMemcpyDeviceToHost);

    printf("Results (first 10 elements):\n");
    for (int i = 0; i < 10; ++i) {
        printf("input[%d]=%.2f, output[%d]=%.2f\n",
```

```

        i, h_input[i], i, h_output[i]));
    }

    cudaFree(d_input);
    cudaFree(d_output);

    return 0;
}

```

1.7 高级用法

1.7.1 编译阶段

编译过程可以分为不同的阶段，您可通过 **clang** (或 **clang++**), **llic**, 或 **lld** 等天数智芯编译器工具执行某个编译阶段。

编译阶段	描述	调用组件	输入文件	输出文件
预处理	编译 C 程序时，编译器在这个阶段进行 <code>#includes</code> 和 <code>#define</code> 宏替换	clang	.c, .cc, .cpp, .cxx	.c, .cc, .cpp, .cxx
生成 LLVM IR	从源文件生成 LLVM IR	clang	.cu	.ll
编译到 OBJ 文件	将 LLVM IR 编译到 OBJ 文件	llic	.ll	.o
从 OBJ 文件生成 CUDA 二进制文件	链接 OBJ 文件以生成 CUDA binary 文件	lld	.o	.cubin
生成 fatbin 文件	从 CUDA binary 文件创建 fatbin 文件	fatbinary	.cubin	.fatbin
汇编	编译主机端代码并且将 fatbin 包入其中生成主机端 OBJ 文件	clang	.c, .cc, .cpp, .cxx, .fatbin	.o
创建可执行文件	链接以创建可执行文件	lld	.o	executable (a.out by default)

术语说明：

- LLVM IR (Intermediate Representation)：LLVM 中间表示，连接着编译器前端和编译器后端。
- lld：是 LLVM 项目的链接器。
- llic：是 LLVM 项目的静态编译器。

1.7.2 输入和输出文件

天数智算软件栈在输入输出文件类型上兼容主流的 GPU 编程模型。

文件扩展名	描述
.cu	CUDA 源文件, 包含主机代码和设备端的函数
.c	C 源代码
.cc, .cxx, .cpp	C++ 源代码
.cubin	单个 GPU 架构的 CUDA 二进制文件
.fatbin	包含.cubin 文件的 CUDA 二进制文件
.o	Obj 文件
.a	库文件
.res	资源文件
.so	共享 Obj 文件

1.7.3 编译步骤示例

示例 可以扩展到下面的编译步骤，适用于编译更复杂的项目。假设使用默认安装路径 `/usr/local/corex/`，请注意执行示例命令需要您对 `/usr/local/corex/` 有写权限。

1.7.3.1 1. 生成 LLVM IR

为 `.cu` 文件中的 CUDA 代码部分生成 LLVM IR。

假设我们要编译 `saxpy.cu`，它包含设备内核和主机部分代码。执行以下命令，为 CUDA 代码部分生成一个名为 `saxpy.ll` 的文本 LLVM IR 文件：

```
$ cd /usr/local/corex/examples/cuda
$ clang++ -O3 -S -emit-llvm --cuda-gpu-arch=ivcore11 --cuda-device-only -std=c++11 -Wall
  ↪ saxpy.cu -o saxpy.ll
```

选项：

- **-S -emit-llvm**: 必选项，生成一个 LLVM IR 文件。
- **--cuda-gpu-arch=ivcore11**: 必选项。针对天玑 150 加速卡或智铠 50 加速卡或智铠 100 加速卡生成文件。
- **-o <LLVM IR filename>**: 可选项，设置生成的 LLVM IR 文件的名称。
- **--cuda-device-only**: 仅生成 CUDA 代码部分。如果省略此选项，还将生成主机部分。

1.7.3.2 2. 将 LLVM IR 编译到 OBJ 文件

使用 LLVM 静态编译器 **llc** 编译 `saxpy.ll`。

```
$ cd /usr/local/corex/examples/cuda
$ llc -march=bi -filetype=obj saxpy.ll -o saxpy.cuda.o
```

选项:

- **-march=bi**: 必选项。针对天数智芯加速卡进行编译。
- **-filetype=obj**: 必选项。生成 OBJ 文件，而不是默认的汇编文件。
- **-o <object filename>**: 可选项。设置输出 OBJ 对象的名称。

1.7.3.3 3. 将 OBJ 文件链接到 .cubin

使用 LLVM 链接器 **lld** 编译 `saxpy.cuda.o`。

```
$ lld -flavor ld.lld --no-warn-missing-entry --no-undefined saxpy.cuda.o -o saxpy.cubin
```

选项:

- **-flavor ld.lld**: 必选项。
- **--no-warn-missing-entry**: 可选项。当 entry 缺失时不提示警告信息。
- **--no-undefined**: 必选项。即使链接器正在创建共享库，也要报告未解析的符号。
- **-o <cubin filename>**: 可选项。设置输出的.cubin 文件名。

1.7.3.4 4. 从 .cubin 文件中创建 .fatbin 文件

使用 **fatbinary** 工具编译 `saxpy.cubin`。

```
$ fatbinary --cuda --64 --image=profile=ivcore11,file=saxpy.cubin --create saxpy.fatbin
```

选项:

- **--cuda**: 指定这是一个 CUDA 编译。
- **--64**: 指定这是一个 64 位的编译。
- **--image=profile=ivcore11,file=saxpy.cubin**: 设置带有配置文件的编译映像列表。语法为 **--image=<keyword>=<string>**，关键字含义如下：
 - **file**: 指定 ELF 文件，是 ld 命令生成的 .cubin 文件。
 - **profile**: 指定架构，设置为 **ivcore11**，代表天垓 150 加速卡或智铠 50 加速卡或智铠 100 加速卡。
- **--create <fatbin filename>**: 必选项。指定输出的 fat 二进制文件名。

1.7.3.5 5. 编译主机部分的代码并链接到 .fatbin

在前面的步骤中，我们只编译 CUDA 代码部分，接下来我们要编译主机部分。

为了从主机端生成 OBJ 文件并从 .fatbin 收集必要的信息，文件 saxpy.fatbin 和 saxpy.cu (使用其主机部分) 都必须包含在以下命令中。

```
$ clang++ -c -std=c++11 -Wall --cuda-host-only -Xclang -fcuda-include-gpubinary -Xclang  
↳ saxpy.fatbin saxpy.cu -o saxpy.o
```

选项:

- **-c -std=c++11**: 使用 C++11 模式。
- **--cuda-host-only**: 忽略代码中的 CUDA 设备部分，只编译主机部分。
- **-Xclang**: 必选项。将 [arg] 值传递给 clang 命令。
- **-fcuda-include-gpubinary**: 通知链接器它将把一个 fat 二进制文件合并到输出 OBJ 文件中。
- **-o <host object filename>**: 设置主机 OBJ 文件名。

1.7.3.6 6. OBJ 文件链接到天数智芯 CUDA 运行环境

这是最后一步。在这一步之后，您将获得名为 saxpy 的可执行文件。

```
$ clang++ -lcudart saxpy.o -o saxpy
```

选项:

- **-lcudart**: 链接 libcudart.so。
- **-o <executable name>**: 设置可执行文件名称。

1.8 编译选项

1.8.1 clang 和 clang++ 选项

Clang 编译器是来自 LLVM 项目，为 C 语言家族，如 C、C++、Objective C/C++、CUDA 等提供了编译器前端和基础工具。**clang** 可用于编译 C 程序，**clang++** 用于编译 C++ 程序。

天数智芯编译器支持所有的 Clang 13 选项。有关 Clang 选项的完整列表，请参阅 [Clang 编译器选项](#)。

以下是一些常用的选项，供您参考。

Clang 选项	描述
--cuda-compile-host-device	为主机和设备编译 CUDA 代码 (默认)。对非 CUDA 编译没有影响。
--cuda-device-only	只为设备编译 CUDA 代码。
--cuda-gpu-arch=< value >	指定目标 CUDA GPU 架构。为 NVIDIA GPU 指定 sm_xy ，或为天垓 150 加速卡或智铠 50 加速卡或智铠 100 加速卡指定 ivcore11 。

Clang 选项	描述
--cuda-host-only	只为主机编译 CUDA 代码。对非 CUDA 的汇编没有影响。
--cuda-noopt-device-debug	启用设备端调试信息生成。
--cuda-path-ignore-env	忽略环境变量以检测 CUDA 安装。
--cuda-path=< value >	指定 CUDA 安装路径。
-cl-single-precision-constant	将双精度浮点常数视为单精度常数。默认已设置，因为天数智芯加速卡对双精度提供有限支持。
-Xcuda-fatbinary < arg >	将 < arg > 参数传递到 fatbinary 调用。
-Xlinker < arg >	将 < arg > 参数传递到链接器。
-x < language >	将后续输入文件视为指定语言的文件。
-fno-strict-aliasing	禁用基于类型的别名分析。
-gdwarf-4	当 LLVM 13 切换为 LLVM 16 后，如果使用 GDB 10.1 以下的版本，为了能使用新的编译器编译出来的程序进行调试，必须加上该选项。
--cubin	编译器将所有 .cu 输入文件编译为仅包含设备代码的 .cubin 文件

天数智算软件栈兼容 Clang 13。更多有关 Clang 13 的用法，请参考 [Clang 13 文档](#)。

Note

天数智芯编译器基于 Clang，Clang 和 nvcc 的默认选项可能存在少量不同，所以编译器选项可能需要略作调整，详见《CUDA 程序迁移指南》。

1.8.2 llc 选项

llc 是 LLVM 项目的静态编译器。它将 LLVM 源输入编译成指定体系结构的汇编语言。天数智算软件栈利用 llc 为 .cu 文件中的 CUDA 代码生成 LLVM IR。

天数智芯编译器支持所有 **llc** 选项。有关更多信息，请参阅 [LLVM 用户指南中 llc 章节](#)。

以下是一些常用的选项，供您参考。

llc 选项	描述
-filetype=< value >	选择文件的类型。 asm 发出一个汇编文件 (.s)。 obj 发出本机 OBJ 文件 (.o)。 null 不发送，供测试性能用。
--iluvatar-spill-sgpr-to-smem	如果子目标支持，如果 Scalar General Purpose Register (SGPR) 已经到达了最大的上限，则对 SGPR 的使用将被溢出到 Shared Memory (SMEM)。

llc 选项	描述
<code>-march=< string ></code>	针对特定架构生成代码。指定针对天数智芯加速卡，您必须设置 <code>-march=bi</code> 。

1.8.3 lld 选项

lld 是 LLVM 项目中的链接器，它是系统链接器的替代，并且运行速度较快。天数智算软件栈利用 **lld** 来链接要生成的 **Obj** 文件和 **cubin** 文件。

天数智芯编译器支持所有的 **lld** 选项。更多信息，请参阅 [LLD 文档](#) 并使用 `lld -flavor ld.lld --help` 查看帮助文档。

以下是一些常用的选项，供您参考。

lld 选项	描述
<code>-flavor ld.lld</code>	调用 Unix 链接器。
<code>--no-undefined</code>	报告无法解析的符号，即使链接器正在创建共享库。
<code>--no-warn-missing-entry</code>	不提示 entry 缺失。
<code>--shared</code>	构建共享对象。

1.8.4 fatbinary 选项

fatbinary 是一个将设备代码组织成“fat 二进制文件”（fat binaries）的工具，这些文件能够保存同一 GPU 源代码的多个编译版本。CUDA fat 二进制文件可能包含 **cubin** 文件。在运行时，当设备功能启动时，CUDA 驱动程序将选择最合适的编译版本。

天数智芯编译器支持所有 **fatbinary** 选项。有关更多信息，请通过 `fatbinary --help` 命令查看帮助内容。

以下是一些常用的选项，供您参考。

fatbinary 选项	描述
<code>--32</code>	说明编译是 32 位的。
<code>--64</code>	说明编译是 64 位的。
<code>--cuda</code>	说明编译的是 CUDA。

fatbinary 选项	描述
--image=< keyword >=< string >	指定带有编译它们的配置文件的映像列表。可用的 keyword 是: file, profile。 file : 指定 ELF 文件, 是 ld 命令生成的 .cubin 文件。 profile : 指定架构。支持设置为 ivcore11, 代表天垓 150 加速卡或智铠 50 加速卡或智铠 100 加速卡。

更多关于 fat 二进制文件的信息, 请参阅 [The CUDA Compilation Trajectory](#)。

1.9 参考

- LLVM 用户指南: [Compiling with clang](#)
- Clang 工具链: [Tools](#)

商标声明

- 天数智芯、天数智芯 logo、Iluvatar CoreX 等商标、标识、组合商标为上海天数智芯半导体有限公司之注册商标或商标，受法律保护。
- CentOS 标识为 Red Hat 公司的商标。
- Docker 为 Docker 公司在美国和其他国家的商标或注册商标。
- Linux 为 Linus Torvalds 在美国和其它国家的注册商标。
- NVIDIA、CUDA、GeForce 和 GeForce GTX 为 NVIDIA 公司在美国和/或其它国家的商标和/或注册商标。
- PyTorch 为 Facebook 公司的商标。
- TensorFlow 为 Google 公司的商标。
- Ubuntu 为 Canonical 公司的注册商标。

除上述明确列出的商标外，本内容中可能存在的任何其他未明示的产品名称、服务名称、企业名称、标识、口号、域名或其他商业标记 (无论是否标注 ™ 或 ® 符号)，均假定归属于其各自合法的商标权人所有。